

Hewlett Packard Enterprise

SAN Unified Platform

Integrated Simulator, BFS Discovery Engine, Graph/Search Database, AI Assistant (RAG & Agent), and Multi-Tenant RBAC Dashboard

Complete User Manual & Operations Directory

An exhaustive, step-by-step systems reference guide mapping every single button, drop-down, input box, canvas interaction, backend API request, and database pipeline across the entire HPE SAN management monorepo.

Prepared For:	HPE SAN Operations & DevOps Engineering Teams
Version:	2.0.0
Document Type:	End-to-End Technical User Manual
Date:	May 19, 2026



Contents

1	Monorepo Architecture & Services	2
1.1	Core Services & Directory Structure	2
1.2	Ontological Data Schema	2
2	Exhaustive Page-by-Page Button & Interaction Directory	3
2.1	Authentication & Tenant Scoped Login Page	3
2.2	Network Discovery Control Page	4
2.3	Interactive Testing & Topology Page	5
2.4	Resource Inventory Explorer Page	7
2.5	Interactive Console Emulator Page	7
2.6	HPE SAN AI Assistant Diagnostic Workspace	8
2.7	System Health Dashboard Page	9
2.8	Admin Control Panel Page	10
3	Under-the-Hood Mechanics & Processing Pipelines	11
3.1	Lightweight OS Device Fingerprinting	11
3.2	The Iterative BFS Network Discovery Crawler	11
3.3	LLM Diagnostic Logic & The Stateful SAN Agent	12

1 Monorepo Architecture & Services

The HPE SAN Unified Platform unifies a virtualized storage network, an automated crawler engine, a graph-based topology viewer, and cutting-edge artificial intelligence agents to streamline SAN system operations.

1.1 Core Services & Directory Structure

The system comprises five core services operating in unison:

1. **React Vite Frontend Dashboard** (Port 3000): Interactive UI using React Flow, Cytoscape visualizations, resizable column grids, live-typing terminal shells, and Web-Socket/SSE log streaming.
2. **Flask Master REST API** (Port 5005): Orchestration layer serving Neo4j data transformations, synthetic database models, Elasticsearch routing, and RAG agent logic.
3. **Python Network Simulator** (Port 5001): Replays production CLI commands, models multipath arrays, and manages an in-process socket routing table.
4. **Node.js + Express Chatbot Service** (Port 5010): Manages user authentication, multi-tenant RBAC permissions, and persistent MongoDB chat history.
5. **Database Cluster:**
 - **Neo4j** (Port 7474): Maps physical/logical relationships (e.g., switches connected to storage node ports).
 - **Elasticsearch** (Port 9200): Drives the indexer engine for high-speed cross-component string searches.
 - **MongoDB** (Port 27017): Retains application schemas, user credentials, and historical LLM chat records.

1.2 Ontological Data Schema

The SAN is modeled as a directed graph representing real-world SAN physical dependencies. The relationships are illustrated in the following structural chain:

ArraySystem $\xrightarrow{\text{HAS_NODE}}$ Node/Controller $\xrightarrow{\text{HAS_PORT}}$ Port $\xrightarrow{\text{CONNECTS_TO}}$ Host HBA $\xrightarrow{\text{CONNECTS_TO}}$ ArraySys

Additionally, array enclosures are modeled hierarchically:

ArraySystem $\xrightarrow{\text{HAS_CAGE}}$ Cage $\xrightarrow{\text{CONTAINS}}$ PhysicalDisk

2 Exhaustive Page-by-Page Button & Interaction Directory

This section provides an absolute, granular mapping of every single button, dropdown selector, check-box, drag divider, terminal shell, and text entry box across all pages in the platform.

2.1 Authentication & Tenant Scoped Login Page

The portal handles user authentication and sets dynamic privilege overrides:

1. Form Toggle Option (“Sign up” / “Sign in” Link):

- **Interaction:** User clicks the blue toggle link.
- **Under the Hood:** Updates local React state `isSignUp` between `true` and `false`.
- **UI Transition:** Switches the form fields dynamically, revealing or hiding additional setup fields (e.g., Team, Managed Teams, Managed Clusters) depending on signup mode.

2. Username/Email Input Field:

- **Interaction:** User types their identifier string.
- **Under the Hood:** Triggers keyboard listener binding updates to local `email` state.
- **UI Transition:** Displays entered text inside the email entry frame.

3. Password Input Field:

- **Interaction:** User types their password securely.
- **Under the Hood:** Updates local `password` state securely.
- **UI Transition:** Characters are masked as black dots.

4. Role Selector Dropdown:

- **Interaction:** User clicks and selects a role from: `Team Member`, `Manager`, `Senior Manager`, or `Admin`.
- **Under the Hood:** Updates the local `role` state.
- **UI Transition:** Selection is displayed in the input field.

5. Team Input Box:

- **Interaction:** User enters their organizational team name (e.g. `team-alpha`).
- **Under the Hood:** Updates local `team` state.
- **UI Transition:** Shows text in place. Used to restrict visibility under RBAC scope configurations.

6. Cluster Input Box:

- **Interaction:** User enters their target cluster namespace (e.g. `cluster-1`).
- **Under the Hood:** Updates local `cluster` state.
- **UI Transition:** Shows text in place. Used to lock structural boundaries.

7. Action Submit Button (“Sign In” / “Create Account”):

- **Interaction:** User clicks the green button.
- **Under the Hood:** Executes a POST request to `/chatbot/auth/login` or `/chatbot/auth/register` on Port 5010. The backend returns a signed JWT, storing the user payload in `AuthContext` and updating local storage context.
- **UI Transition:** Navigates the browser to the main dashboard.

2.2 Network Discovery Control Page

Tracks, starts, or completely wipes crawling iterations:

1. View Mode Switcher (Graph View Button):

- **Interaction:** User clicks the “Graph” tab.
- **Under the Hood:** Sets local state `viewMode` to `"graph"`.
- **UI Transition:** Renders the React Flow canvas (`TopologyCanvas.jsx`) mapping discovered nodes and edges.

2. View Mode Switcher (Neural View Button):

- **Interaction:** User clicks the “Neural” tab.
- **Under the Hood:** Sets local state `viewMode` to `"neural"`.
- **UI Transition:** Renders a custom force-directed spring diagram (`NeuralGraph.jsx`) using vector mathematics to cluster devices.

3. View Mode Switcher (Grid View Button):

- **Interaction:** User clicks the “Grid” tab.
- **Under the Hood:** Sets local state `viewMode` to `"grid"`.
- **UI Transition:** Swaps the display layout into structured card tiles detailing device properties.

4. Wipe Database Button (“Wipe”):

- **Interaction:** User clicks the red wipe button.
- **Under the Hood:** Issues a POST request to `/api/graph/wipe` on Port 5005, which triggers the Cypher query `MATCH (n) DETACH DELETE n` to completely clear the Neo4j database, and updates the local state.
- **UI Transition:** Clears the visual map, returning the interface to an empty state.

5. Upload Log Button (“Log”):

- **Interaction:** User clicks the upload log button.
- **Under the Hood:** Launches the native browser file explorer modal. Upon choosing a terminal log (`.txt`), the client submits it via a POST request to `/api/discover/ingest` on Port 5005.
- **UI Transition:** Initiates automatic parser processing, registering a new discovered device dynamically on the dashboard.

6. Start Discovery Button (“Start Discovery”):

- **Interaction:** User clicks the start discovery button.
- **Under the Hood:** Sends a POST request to `/api/discover` with seed IP `10.20.10.5`, spawning a background thread in the Flask server. It then initializes a live `EventSource` connection to `/api/discover/stream` for Server-Sent Events (SSE).
- **UI Transition:** Spawns a running indicator badge and slides open the live terminal logs stream panel.

7. Cancel Discovery Button (“Cancel Discovery”):

- **Interaction:** User clicks the cancel discovery button.

- **Under the Hood:** Sends a POST request to `/api/discover/cancel` to safely halt the background BFS crawler thread.
- **UI Transition:** Cancel button vanishes and the running spinner halts.

8. Log Panel Toggle Button (“Log”):

- **Interaction:** User clicks the log toggle pill.
- **Under the Hood:** Swaps the local `showLogs` state.
- **UI Transition:** Collapses or expands the right-side JSON logging console panel.

9. Refresh Button (“Refresh”):

- **Interaction:** User clicks the refresh button.
- **Under the Hood:** Sends a GET request to `/api/graph/neo4j` to fetch the latest nodes and edges.
- **UI Transition:** Canvas flashes briefly and updates with the latest topological status.

2.3 Interactive Testing & Topology Page

The primary interface for testing node status, zoning, and decommission scopes:

1. Secondary Sub-Tab Switcher (“SAN Diagram” Button):

- **Interaction:** User clicks the tab labeled “SAN Diagram”.
- **Under the Hood:** Updates the local `activeTab` React state to `"diagram"`.
- **UI Transition:** Displays the resizable 3-column SAN grid.

2. Secondary Sub-Tab Switcher (“Visual Map” Button):

- **Interaction:** User clicks the tab labeled “Visual Map”.
- **Under the Hood:** Updates the local `activeTab` React state to `"visual"`.
- **UI Transition:** Renders the dynamic SVG React Flow mapping canvas.

3. Secondary Sub-Tab Switcher (“Decommissioned” Button):

- **Interaction:** User clicks the tab labeled “Decommissioned”.
- **Under the Hood:** Updates the local `activeTab` React state to `"decommissioned"`.
- **UI Transition:** Lists all greyed-out components that have been decommissioned.

4. Search Bar Input Box:

- **Interaction:** User types text (e.g. Brocade or 10.20.10.1).
- **Under the Hood:** Triggers keyboard listener binding updates to local `searchQuery` state.
- **UI Transition:** Instantly filters the visible card grid. Walks up parent ID paths so parent cages remain visible if children match.

5. Column Resizing Dividers (Drag Handles):

- **Interaction:** User clicks and drags the vertical boundary lines between columns in the SAN Diagram layout.
- **Under the Hood:** Captures mouse move drag events, updating the `colWidths` array state (e.g., `[350, 350, 350]`) based on drag displacement.

- **UI Transition:** Instantly resizes the column widths for Arrays, Switches, and Hosts.

6. Node Checkboxes:

- **Interaction:** User clicks the checkbox located inside any node card.
- **Under the Hood:** Appends or removes the clicked node ID from the local `selectedIds` Set state.
- **UI Transition:** Highlights all active connected data paths associated with the checked nodes on the canvas.

7. Clear Selection Button:

- **Interaction:** User clicks the “Clear Selection” button (appears only when selections exist).
- **Under the Hood:** Resets the active `selectedIds` state to a clean set.
- **UI Transition:** Clears all selection highlights and resets the checkboxes.

8. Export Excel Button (“Export Excel”):

- **Interaction:** User clicks the download icon button.
- **Under the Hood:** Aggregates discovered nodes and edges. Runs the client-side sheetjs workbook compilation utility, generating a formatted multi-sheet Excel spreadsheet (.xlsx).
- **UI Transition:** Downloads the generated file to the user’s local system.

9. Import Config Button:

- **Interaction:** User clicks the “Import Config” button.
- **Under the Hood:** Sets `showImport` state to `true`, launching the file picker overlay.
- **UI Transition:** Opens the modal backdrop.

10. Simulate Role Dropdown (Admin-Only):

- **Interaction:** User clicks and selects a simulated role (Admin, Manager, Team Member).
- **Under the Hood:** Dynamically overrides local state variables inside `AuthContext`.
- **UI Transition:** Instantly updates the visible layout, showing how manager or team member privilege tiers restrict data access.

11. Team Selector Dropdown (Admin & Manager Only):

- **Interaction:** User clicks and selects an active organizational team.
- **Under the Hood:** Updates the local `selectedTeamId` state.
- **UI Transition:** Automatically restricts the visible canvas to only show devices mapped to the selected team’s cluster.

12. Decommission Toggle Switch (inside NodeCard sidebar):

- **Interaction:** User toggles the decommission switch inside the selected node’s card details.
- **Under the Hood:** Triggers a PATCH request to `/api/ontology/nodes/<id>` on Port 5005, updating the node’s decommissioned status.

- **UI Transition:** Node immediately turns grey, vanishes from the standard canvas, and is moved to the decommissioned tab.

13. Property Input Editors (inside NodeCard sidebar):

- **Interaction:** User types inside property input boxes (e.g. Rack Location, Notes) and presses enter or loses focus.
- **Under the Hood:** Sends a PATCH request to `/api/ontology/nodes/<id>` on Port 5005 to update Neo4j attributes.
- **UI Transition:** Persists updates to the node details, updating the node properties panel.

2.4 Resource Inventory Explorer Page

Provides nested tree audits of active device categories:

1. HierarchyTree Collapsible Folders:

- **Interaction:** User clicks the folder triangle icons next to parent devices.
- **Under the Hood:** Updates local state arrays tracking expanded node keys.
- **UI Transition:** Smoothly unfolds child arrays, cages, switches, and disks.

2. Search Query Box:

- **Interaction:** User types inside the inventory search box.
- **Under the Hood:** Updates local filtering state.
- **UI Transition:** Dynamically filters the tree, showing matching devices while keeping their parent hierarchies visible.

3. RBAC Scope override Panel (Admin/Manager):

- **Interaction:** User clicks simulate buttons in the inventory header.
- **Under the Hood:** Sets simulation scope variables.
- **UI Transition:** Instantly expands or shrinks visible nodes in the tree based on the selected tenant role's permissions.

2.5 Interactive Console Emulator Page

Mimics a real jump-host server and interactive terminal logins:

1. Device IP Sidebar Selection:

- **Interaction:** User clicks a virtual IP in the sidebar list.
- **Under the Hood:** Initiates the SSH connection sequence for that host.
- **UI Transition:** Opens the terminal console, displays the initial login prompt, and requests confirmation for the simulated host key.

2. Double-Click Node Terminal Modal:

- **Interaction:** User double-clicks any node card on the Discovery or Topology canvases.
- **Under the Hood:** Launches a Cisco Packet Tracer-style terminal modal overlay.

- **UI Transition:** Opens the modal terminal overlay directly on the canvas, initiating the SSH login handshake.

3. SSH key Validation Confirm Input (“yes/no”):

- **Interaction:** User types **yes** and presses Enter.
- **Under the Hood:** Confirms simulated host key validation.
- **UI Transition:** Moves the terminal sequence forward to the password prompt.

4. SSH Password Field Typing:

- **Interaction:** User types password strings and presses Enter.
- **Under the Hood:** Captures keyboard focus via a hidden input element. Masking prevents any password characters from being written to the terminal buffer.
- **UI Transition:** No characters or stars are rendered on screen (silent password masking), mimicking authentic Unix SSH terminals. Completing login opens the command prompt (e.g. `cli%` or `$`).

5. Contextual Command Quick Pills:

- **Interaction:** User clicks any pill button (e.g. `showsys`, `fabricshow`, `multipath -ll`).
- **Under the Hood:** Sends a POST request to `/api/sim/exec` on Port 5005 containing the target command.
- **UI Transition:** Automatically inputs the selected command into the prompt, executes it immediately, and appends the simulated device output to the terminal window.

6. Clear Output Button:

- **Interaction:** User clicks the “Clear” button.
- **Under the Hood:** Resets the local terminal buffer array.
- **UI Transition:** Clears the console output screen, resetting the cursor to the top line.

7. Ask AI Pill (“Ask AI about...”):

- **Interaction:** User clicks the Ask AI pill.
- **Under the Hood:** Saves the active host’s IP and terminal state to session storage as a diagnostic context hint.
- **UI Transition:** Navigates the browser to the RAG Chat Page, prepopulating the AI text box with the diagnostic query.

2.6 HPE SAN AI Assistant Diagnostic Workspace

An LLM diagnostic dashboard for RAG and GraphRAG:

1. New Chat Button (“New Chat”):

- **Interaction:** User clicks the green New Chat button.
- **Under the Hood:** Resets chat variables and clears active thread states.
- **UI Transition:** Clears the chat window, ready to start a new diagnostic session.

2. Chat History List Sidebar Items:

- **Interaction:** User clicks a past chat thread title in the sidebar list.
- **Under the Hood:** Sends a GET request to `/chatbot/chat/<id>` on Port 5010 to fetch historical messages.
- **UI Transition:** Loads the historical messages and updates the chat layout.

3. Delete Chat Button (Trash Can Icon):

- **Interaction:** User clicks the trash can button next to a thread title.
- **Under the Hood:** Sends a DELETE request to `/chatbot/chat/<id>` on Port 5010 to remove the thread from MongoDB.
- **UI Transition:** Thread is removed from the sidebar history list.

4. Categorized Query Radial Button (“Queries”):

- **Interaction:** User clicks the circular queries radial button.
- **Under the Hood:** Updates the local popover state to `true`.
- **UI Transition:** Opens a popover menu showing query presets categorized under **Arrays**, **Hosts**, and **Network**. Clicking any query template auto-fills the message text box.

5. AI Engine Mode Tabs (“SAN Agent” / “Standard RAG” / “GraphRAG”):

- **Interaction:** User clicks one of the three engine tabs.
- **Under the Hood:** Updates the local `aiEngine` state.
- **UI Transition:** Switches active LLM routing on submit (SAN Agent runs multithreaded CLI loop; Standard RAG uses standard system prompts; GraphRAG translates questions into Cypher queries).

6. Show/Hide Analysis Toggle Switch:

- **Interaction:** User toggles the analysis switch.
- **Under the Hood:** Updates the local `showSidebar` state.
- **UI Transition:** Expands or collapses the right-side reasoning panel showing raw CLI execution traces.

2.7 System Health Dashboard Page

A high-level health overview mapping active alerts:

1. Refresh Button:

- **Interaction:** User clicks the refresh icon.
- **Under the Hood:** Sends GET requests to both Port 5005 and Port 5010 to pull the latest hardware metrics and service statuses.
- **UI Transition:** Updates metric cards, service online/offline indicators, and active alert lists.

2. Failed Nodes Alert Item:

- **Interaction:** User clicks a failed node alert list item.
- **Under the Hood:** Saves target device variables in session storage context.
- **UI Transition:** Navigates the browser to the Topology tab, auto-focusing the degraded device in the sidebar.

2.8 Admin Control Panel Page

Provides database CRUD and fake data generators:

1. Generate Topology Button:

- **Interaction:** User enters parameters (Arrays, Switches, Hosts) and clicks “Generate Topology”.
- **Under the Hood:** Sends a POST request to `/api/faker/san` on Port 5005 to generate synthetic SAN topology objects.
- **UI Transition:** Displays the generated JSON preview, updating download links.

2. Download JSON Button:

- **Interaction:** User clicks “Download JSON”.
- **Under the Hood:** Assembles a client-side Blob containing the generated topology schema.
- **UI Transition:** Downloads the generated configuration file to the user’s system.

3. Import to Neo4j Button:

- **Interaction:** User clicks “Import to Neo4j”.
- **Under the Hood:** Sends a POST request containing the synthetic JSON payload to Port 5005, writing nodes and edge relationships directly into the live graph database.
- **UI Transition:** Refreshes the database state indicator.

4. Add Node Button:

- **Interaction:** User completes the Node Creation form and clicks “Add Node”.
- **Under the Hood:** Sends a POST request containing properties to `/api/graph/nodes` on Port 5005.
- **UI Transition:** Clears input fields, returning a successful creation notification.

5. CSV Ingest Upload:

- **Interaction:** User uploads a CSV spreadsheet.
- **Under the Hood:** Submits spreadsheet data to `/api/ingest/spreadsheet` to batch-import nodes.
- **UI Transition:** Displays progress bar and success count.

6. Save Schema Button:

- **Interaction:** User updates whitelisted properties in the Field Schema editor and clicks “Save Schema”.
- **Under the Hood:** Sends a PUT request to update `field_definitions.json` on Port 5005.
- **UI Transition:** Updates validation checks, saving changes in the Master API.

3 Under-the-Hood Mechanics & Processing Pipelines

3.1 Lightweight OS Device Fingerprinting

During discovery crawl runs, the system classification script identifies the vendor hardware before executing queries:

```

1 def fingerprint_device(ip: str, virtual_network) -> DeviceType:
2     meta = virtual_network.get_metadata(ip)
3     if meta.get("type") == "array":
4         return DeviceType.HPE_ARRAY
5     if meta.get("type") == "switch":
6         return DeviceType.HPE_ARRAY
7     if meta.get("type") == "host":
8         if meta.get("os_type") == "windows":
9             return DeviceType.WINDOWS
10        return DeviceType.LINUX
11
12    # Active Shell Probing Fallback
13    probe = virtual_network.execute(ip, "showsys")
14    if probe and "Connection refused" not in probe:
15        if "TotalCap" in probe or "AllocCap" in probe:
16            return DeviceType.HPE_ARRAY
17
18    probe = virtual_network.execute(ip, "uname -a")
19    if probe and "Linux" in probe:
20        return DeviceType.LINUX
21
22    return DeviceType.UNKNOWN

```

Listing 1: Fingerprint Classification Pipeline

3.2 The Iterative BFS Network Discovery Crawler

When a user launches the discovery engine, the crawler utilizes an iterative BFS traversal pattern:

```

1 class DiscoveryCrawler:
2     def discover(self, seed_ips: List[str]):
3         self.running = True
4         self.visited = set()
5         self.queue = deque(seed_ips)
6
7         while self.queue:
8             ip = self.queue.popleft()
9             if ip in self.visited:
10                continue
11            self.visited.add(ip)
12
13            # Fingerprint and execute commands
14            dev_type = fingerprint_device(ip)
15            if dev_type == "hpe_array":
16                outputs = self._run_commands(ip, HPE_COMMANDS)
17                parsed = parse_sim_array_output(outputs)
18                # Store in databases
19                self.neo4j.store(parsed)
20                self.es.index(parsed)
21                self.mongo.store(parsed)

```

```
22         # Extract and enqueue newly found IPs
23         new_ips = self._extract_linked_ips(parsed)
24         for new_ip in new_ips:
25             if new_ip not in self.visited:
26                 self.queue.append(new_ip)
```

Listing 2: Discovery BFS Core Pipeline

3.3 LLM Diagnostic Logic & The Stateful SAN Agent

When executing the **SAN Agent** mode in the AI Assistant, the platform deploys a stateful reasoning loop:

1. **Plan Formulation:** The LLM evaluates the user's natural language query and determines which CLI commands (e.g., `showsys`, `showhost`) need to run on the target IP array.
2. **Tool Execution:** The agent connects to the Simulator's CLI engine and executes the planned commands.
3. **Real-Time Parsing:** Outputs are run through specialized regex parsers to extract active entities (e.g., degraded disks, host WWNs).
4. **Graph Enrichment:** The parsed entities are upserted into Neo4j to ensure the topology graph is in sync with the live device state.
5. **Answer Formulation:** The LLM synthesizes the diagnostic timeline and the final answer, streaming the entire trace back to the user.